МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«МИРЭА — Российский технологический университет»

РТУ МИРЭА

Институт технологий управления

Центр технологий 1С

ОТЧЁТ О ПРАКТИЧЕСКОМ ЗАНЯТИИ № 3

по дисциплине «Проектирование и архитектура программного обеспечения»

АРХИТЕКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ И ПРИМЕНЕНИЕ ПАТТЕРНОВ ПРОЕКТИРОВАНИЯ

«Персональный тайм-трекер на Java»

Выполнил: студент группы ЭПБО-01-25
Хусяинов Э. Р.

Проверил: Яковлев Михаил Сергеевич

Москва — 2026

СОДЕРЖАНИЕ

1 Назначение документа и связь с практической работой № 2

2 Архитектурные принципы

3 Выбор паттернов проектирования

4 Применение паттернов проектирования

4.1 State (Состояние)

4.2 Observer (Наблюдатель)

4.3 Strategy (Стратегия)

4.4 Adapter (Адаптер)

4.5 Factory Method (Фабричный метод)

5 Дополнительные архитектурные решения

6 Выбор и настройка системы сборки

7 Структура проекта

8 Сборка и запуск

9 Границы текущей версии и план дальнейшей работы

Заключение

Список использованных источников

1 НАЗНАЧЕНИЕ ДОКУМЕНТА И СВЯЗЬ С ПРАКТИЧЕСКОЙ РАБОТОЙ № 2

Настоящий документ дополняет спецификацию требований к программному обеспечению (SRS) «Персональный тайм-трекер на Java», подготовленную в практической работе № 2. Цель практической работы № 3 — обосновать выбор паттернов проектирования и показать их применение на конкретных классах Java, а также подготовить промышленную структуру проекта и настроить систему сборки.

В качестве исходной предметной области сохранён персональный сценарий: один пользователь создаёт задачи по категориям, вручную запускает и останавливает таймер, фиксирует причины пауз, а затем анализирует фактическое время. Приложение хранит данные локально и не требует учётной записи или сетевого сервера.

Требования SRS, на которые непосредственно опирается архитектура: ФТ-06—ФТ-12 (управление сеансом и паузами), ФТ-13—ФТ-17 (GUI, фон, уведомления), ФТ-18—ФТ-26 (аналитика и отчёты), ФТ-27—ФТ-28 (локальное хранение и обработка ошибок). Проектные решения не подменяют требования заказчика: числовые пороги рекомендаций и детали интеграции с ОС остаются настраиваемыми параметрами.

2 АРХИТЕКТУРНЫЕ ПРИНЦИПЫ

Для ограничения связанности выбрана слоистая архитектура с разделением на представление, прикладной слой, доменную модель и инфраструктуру. Представление отвечает за взаимодействие с пользователем; прикладной слой реализует сценарии; доменная модель содержит предметные сущности и правила; инфраструктура реализует технические механизмы хранения и интеграции с рабочим столом.

Основное правило зависимостей: прикладная логика обращается к абстракциям доменного слоя, а не к конкретным классам SQLite, AWT или Swing. Например, TimerService зависит от WorkSessionRepository, а конкретное сохранение выполняет SqliteWorkSessionRepository. Благодаря этому реализацию хранилища можно заменить без переписывания таймера.

Сущности предметной области: Category, Task, WorkSession и PauseRecord. Сервисы: TaskService, TimerService, RecommendationService и ReportService. В модели времени активная длительность вычисляется только по рабочим интервалам; паузы хранятся отдельно и не включаются в активное время — это сохраняет правило расчёта из SRS.



Рисунок 1 — Слоистая архитектура и границы паттернов проектирования

3 ВЫБОР ПАТТЕРНОВ ПРОЕКТИРОВАНИЯ

Паттерны отбирались не по принципу максимального количества, а по наличию конкретной повторяющейся задачи в проекте. В лекции № 3 паттерн рассматривается через контекст, проблему, решение и последствия; отдельно подчёркивается риск избыточного применения паттернов. Поэтому в проект включены только те решения, которые снижают связанность или изолируют ожидаемые изменения.

Паттерн и категория	**Классы проекта**	**Решаемая задача**
State — поведенческий	TimerService, TimerState, IdleState, RunningState, PausedState, FinishedState	Контроль допустимых переходов таймера
Observer — поведенческий	TimerObserver, MainWindow, MiniTimerWindow	Рассылка актуального состояния таймера нескольким представлениям
Strategy — поведенческий	RecommendationStrategy, OvertimeRecommendationStrategy, FrequentPauseRecommendationStrategy	Независимые алгоритмы аналитических рекомендаций
Adapter — структурный	DesktopIntegration, AwtDesktopIntegrationAdapter, NoOpDesktopIntegration	Изоляция API системного трея и резервный режим
Factory Method — порождающий	ReportExporterFactory, CsvReportExporterFactory, PdfReportExporterFactory	Создание экспортёра по выбранному формату

4 ПРИМЕНЕНИЕ ПАТТЕРНОВ ПРОЕКТИРОВАНИЯ

4.1 Паттерн State (Состояние)

Проблема. Таймер имеет несколько режимов, и допустимость команды зависит от текущего режима. Например, продолжить можно только после паузы; поставить на паузу можно только работающий сеанс; одновременно нельзя запустить два сеанса. Если проверять все комбинации через набор вложенных if/switch в одном методе, переходы быстро становятся трудными для проверки и расширения.

Решение. TimerService выступает контекстом и хранит ссылку на объект TimerState. Интерфейс TimerState объявляет операции start, pause, resume и stop. Конкретные состояния IdleState, RunningState, PausedState и FinishedState определяют допустимые действия. Состояние не выполняет работу с базой самостоятельно: оно делегирует переход контексту через методы startInternal, pauseInternal, resumeInternal и stopInternal.

Взаимодействие. При нажатии «Старт» MainWindow вызывает TimerService.start(task). Контекст передаёт команду текущему состоянию. Если состояние IdleState или FinishedState, создаётся новый сеанс и устанавливается RunningState. Команда «Пауза» в RunningState фиксирует активный интервал и переводит таймер в PausedState. Команда «Продолжить» в PausedState закрывает запись паузы и возвращает RunningState. Команда «Стоп» завершает сеанс как из RunningState, так и из PausedState, сохраняет WorkSession и переводит контекст в FinishedState.

Польза. Запрещённые переходы сосредоточены в объектах состояний; TimerService не разрастается условными ветвлениями. Для тестирования в практической работе № 4 можно отдельно проверить каждую разрешённую и запрещённую команду. Цена решения — дополнительные классы состояний, оправданные количеством переходов и требованиями ФТ-06—ФТ-12.

4.2 Паттерн Observer (Наблюдатель)

Проблема. Одни и те же данные таймера должны отображаться в главном окне и в мини-панели, а в дальнейшем использоваться для предупреждений. Таймер не должен напрямую обращаться к конкретным Swing-компонентам: иначе предметная логика будет зависеть от интерфейса и от способа отображения.

Решение. TimerService является источником событий и хранит список подписчиков TimerObserver. Подписчик получает неизменяемый снимок TimerSnapshot со статусом, названием задачи, активным временем и плановой длительностью. MainWindow и MiniTimerWindow реализуют TimerObserver и обновляют свои элементы интерфейса при получении нового снимка.

Взаимодействие. После старта, паузы, продолжения, завершения и при периодическом обновлении таймера TimerService формирует TimerSnapshot и уведомляет подписчиков. UI-подписчики переводят обновление на поток Swing через SwingUtilities.invokeLater. Ошибка одного наблюдателя не должна прерывать работу таймера и доставку события остальным наблюдателям.

Польза. Таймер не знает, сколько окон подписано на обновления и как они устроены. Можно добавить виджет, журнал событий или компонент уведомлений, не меняя алгоритм учёта времени. Паттерн напрямую поддерживает требования ФТ-14—ФТ-17.

4.3 Паттерн Strategy (Стратегия)

Проблема. Аналитика должна формировать советы по разным основаниям: систематическое превышение планового времени, частые паузы и другие выявленные закономерности. Если все правила поместить в один крупный метод, добавление нового правила будет затрагивать существующую логику и усложнит проверку.

Решение. RecommendationStrategy задаёт единый контракт evaluate(AnalyticsInput). Отдельные классы OvertimeRecommendationStrategy и FrequentPauseRecommendationStrategy реализуют независимые алгоритмы. RecommendationService хранит список стратегий, последовательно передаёт каждой одинаковый AnalyticsInput и объединяет полученные рекомендации.

Взаимодействие. ReportService или аналитический сценарий передаёт в RecommendationService агрегированные показатели задачи: план, факт, количество и длительность пауз. Сервис вызывает каждую стратегию. Если условие стратегии выполнено, возвращается Recommendation с названием проблемы, объяснением и действием для пользователя. Новое правило добавляется отдельным классом, не требуя изменения уже работающих стратегий.

Польза. Алгоритмы изолированы и воспроизводимы; пороги можно конфигурировать и проверять тестами. Порог «три паузы» и процент превышения в начальной версии являются проектными настройками, а не дословно заданными заказчиком числами. Это согласуется с разделом 8.2 SRS, где такие значения отмечены как допущения.

4.4 Паттерн Adapter (Адаптер)

Проблема. Работа с системным треем и уведомлениями зависит от графической среды ОС. Поддержка SystemTray может отсутствовать в отдельных окружениях Linux, на виртуальных машинах или при запуске без полноценного рабочего стола. Прямая работа MainWindow с AWT SystemTray сделала бы интерфейс зависимым от конкретного API.

Решение. Введён целевой интерфейс DesktopIntegration с операциями attach, showNotification, isAvailable и close. AwtDesktopIntegrationAdapter преобразует вызовы этого интерфейса в операции AWT SystemTray и TrayIcon. NoOpDesktopIntegration предоставляет безопасный резервный вариант, когда системный трей недоступен. DesktopIntegrationFactory выбирает подходящую реализацию при запуске.

Взаимодействие. App создаёт DesktopIntegrationFactory и передаёт выбранную реализацию в MainWindow. Окно вызывает только методы DesktopIntegration и не знает о классах TrayIcon или SystemTray. Если трей не поддерживается, приложение продолжает работать через основное окно, а функциональность сворачивания в трей недоступна без аварийного завершения.

Польза. Интеграция с ОС отделена от интерфейса и бизнес-логики. Это также учитывает замечание преподавателя: в SRS нельзя фиксировать Windows как единственную возможную платформу. Предлагаемая редакция НФТ-02: «Приложение должно запускаться в операционных системах, поддерживающих JDK 17 или новее и Java Desktop. Поддержка Windows, Linux и macOS подтверждается отдельной приёмочной проверкой; недоступность SystemTray не должна блокировать основной сценарий работы». Swing/AWT являются кроссплатформенными API Java, но фактическая доступность трея и уведомлений должна проверяться на целевых ОС; корректное поведение fallback обязательно.

4.5 Паттерн Factory Method (Фабричный метод)

Проблема. Пользователь должен экспортировать отчёт как минимум в PDF и CSV. Если ReportService самостоятельно создаёт конкретные классы экспортёров и содержит их конструкторы, добавление нового формата будет менять сервис и увеличивать его связанность с библиотеками экспорта.

Решение. ReportExporter задаёт общий интерфейс export. Абстрактный создатель ReportExporterFactory содержит операцию export и фабричный метод createExporter. Конкретные создатели CsvReportExporterFactory и PdfReportExporterFactory возвращают CsvReportExporter и PdfReportExporter. ReportService выбирает создателя по ExportFormat, не работая с деталями файлового формата.

Взаимодействие. Пользователь выбирает формат. ReportService получает ReportSnapshot и целевой путь, определяет CSV или PDF, создаёт соответствующую фабрику и вызывает её метод export. CSV формируется средствами Java NIO; PDF — библиотекой OpenPDF. Для добавления нового формата достаточно реализовать новый экспортёр и конкретную фабрику, затем расширить выбор формата.

Польза. Логика выбора и алгоритм формирования файла разделены. Библиотека OpenPDF изолирована в инфраструктуре экспорта и не проникает в доменную модель. Это поддерживает ФТ-25 и упрощает последующее тестирование экспортёров.



Рисунок 2 — Участники выбранных паттернов проектирования

5 ДОПОЛНИТЕЛЬНЫЕ АРХИТЕКТУРНЫЕ РЕШЕНИЯ

Repository. CategoryRepository, TaskRepository и WorkSessionRepository являются контрактами доступа к данным. SqliteCategoryRepository, SqliteTaskRepository и SqliteWorkSessionRepository реализуют эти контракты через JDBC. Repository — архитектурный шаблон доступа к данным, а не один из пяти перечисленных GoF-паттернов. Это различие важно для корректной терминологии.

Разделение слоёв. Пакет domain не должен импортировать Swing, AWT или SQLite. Пакет application работает с доменными объектами и интерфейсами репозиториев. Пакет infrastructure содержит технические реализации. Пакет presentation отвечает за окна и действия пользователя. Такой подход позволяет в практической работе № 4 тестировать правила времени без поднятия графического интерфейса.

Платформенная независимость. Путь к файлу БД строится относительно системного свойства user.home, а не фиксированного диска. Взаимодействие с системным треем скрыто за DesktopIntegration. Поэтому архитектура не объявляет Windows единственной поддерживаемой ОС. Приёмочная проверка кроссплатформенности должна включать сборку и запуск на Windows, Linux и macOS при наличии соответствующей среды Java Desktop.

6 ВЫБОР И НАСТРОЙКА СИСТЕМЫ СБОРКИ

Для проекта выбрана Apache Maven. Согласно лекции № 4, Maven является декларативным инструментом: в pom.xml описываются координаты проекта, версия Java, зависимости и плагины, а сам Maven выполняет стандартный жизненный цикл. Для учебного проекта это уменьшает объём ручной конфигурации и делает структуру предсказуемой.

Координаты проекта: groupId — ru.mirea.khusyainov; artifactId — personal-time-tracker; version — 1.0.0; packaging — jar. Целевая версия компиляции — Java 17. Исходный код размещён в src/main/java, ресурсы — в src/main/resources, тесты — в src/test/java, результат сборки — в target.

Зависимости: sqlite-jdbc обеспечивает драйвер локальной SQLite; OpenPDF используется только на границе экспорта PDF; JUnit Jupiter подключён с областью test для последующей практической работы № 4. Зависимость JUnit не попадает в runtime-состав исполняемого приложения.

Плагин maven-compiler-plugin задаёт release 17; maven-surefire-plugin запускает тесты JUnit Platform; maven-shade-plugin формирует исполняемый JAR со сторонними runtime-зависимостями и указывает главный класс App в манифесте. Включён ServicesResourceTransformer, чтобы не потерять записи SPI сторонних библиотек при упаковке.

7 СТРУКТУРА ПРОЕКТА

Корневой каталог personal-time-tracker содержит pom.xml, README.md и .gitignore. В src/main/java код разделён на пакеты ru.mirea.khusyainov.timetracker.domain, application, infrastructure и presentation. Главный класс App находится в корневом пакете приложения и отвечает только за сборку зависимостей и запуск Swing-интерфейса.

В domain/model размещены Category, Task, WorkSession, PauseRecord и TimerStatus. В domain/repository находятся интерфейсы репозиториев. В application/task расположен TaskService; application/timer содержит TimerService и классы состояний; application/analytics содержит стратегии рекомендаций; application/report содержит фабрики и экспортёры. В infrastructure/persistence находятся SQLite-реализации, а infrastructure/platform содержит адаптер рабочего стола. В presentation находятся MainWindow и MiniTimerWindow.

README.md содержит назначение приложения, технологический стек, схему каталогов, инструкции по сборке и запуску, а также список применённых паттернов. Каталог target не включается в репозиторий: это воспроизводимый результат сборки.

8 СБОРКА И ЗАПУСК

Предварительные условия: установлен JDK 17 или новее; установлен Apache Maven 3.9 или новее; при первом запуске доступен Maven Central для загрузки зависимостей. Откройте терминал в каталоге personal-time-tracker.

Команда mvn clean package удаляет предыдущий каталог target, проверяет проект, компилирует исходный код, запускает имеющиеся тесты и упаковывает исполняемый JAR. После успешной сборки запуск выполняется командой java -jar target/personal-time-tracker.jar.

На первом запуске приложение создаёт файл базы данных в каталоге .personal-time-tracker внутри домашнего каталога пользователя. Для проверки базового сценария создайте задачу, задайте категорию и плановую длительность, нажмите «Старт», затем «Пауза» и укажите причину, нажмите «Продолжить» и завершите сеанс кнопкой «Стоп». Сеанс и паузы сохраняются локально.

В среде без поддержки SystemTray приложение должно оставаться доступным через основное окно. Мини-панель является отдельным Swing-окном. Реальная проверка уведомлений и поведения при сворачивании должна выполняться на целевой графической среде; headless-режим не является полноценной средой для проверки GUI.

9 ГРАНИЦЫ ТЕКУЩЕЙ ВЕРСИИ И ПЛАН ДАЛЬНЕЙШЕЙ РАБОТЫ

Представленная версия является промежуточной реализацией для демонстрации архитектуры, паттернов и системы сборки. Она реализует создание категорий по имени при создании задачи, локальное сохранение задач, базовое управление таймером, фиксацию причин пауз, SQLite-репозитории, мини-панель, адаптер системного трея и классы экспорта/рекомендаций.

Полная аналитика по произвольным периодам, фильтры по категориям и задачам, круговые диаграммы, история сохранённых снимков отчётов, восстановление незавершённого сеанса после перезапуска, точные alert-пороги и полный набор пользовательских настроек не заявляются как завершённые функции текущей версии. Они остаются предметом дальнейших этапов и должны реализовываться в соответствии с SRS.

Практическая работа № 4 должна добавить unit-тесты основных методов и интеграционные сценарии JUnit 5. Особое внимание следует уделить допустимым переходам состояний таймера, исключению времени пауз из активного времени, транзакционному сохранению сеансов, поведению стратегий на граничных значениях и экспорту файлов. Практическая работа № 5 должна добавить многостадийный Dockerfile, сборку и запуск тестов в Linux-контейнере. При этом графический интерфейс и системный трей должны оставаться необязательными для контейнерного режима.

ЗАКЛЮЧЕНИЕ

Для персонального тайм-трекера выбрана слоистая архитектура с отделением предметной модели, прикладных сценариев, представления и инфраструктуры. Паттерны State, Observer, Strategy, Adapter и Factory Method применены к конкретным проблемам проекта: управлению состояниями таймера, публикации обновлений, независимым алгоритмам рекомендаций, интеграции с рабочим столом и экспорту отчётов.

Проект организован как Maven-приложение на Java 17 с логическими пакетами, зависимостями SQLite JDBC, OpenPDF и JUnit 5, а также конфигурацией исполняемого JAR. Структура подготовлена для последующего расширения тестами и контейнеризацией. Архитектура не фиксирует Windows как единственную платформу и предусматривает резервное поведение при недоступности системного трея.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Яковлев М. С. Лекция № 3 «Паттерны проектирования». — МИРЭА — Российский технологический университет, 2026.

2. Яковлев М. С. Лекция № 4 «Системы сборки». — МИРЭА — Российский технологический университет, 2026.

3. Практическое занятие № 3 «Паттерны проектирования. Системы сборки» по дисциплине «Проектирование и архитектура программного обеспечения». — 2026.

4. Хусяинов Э. Р. Спецификация требований к программному обеспечению (SRS) «Персональный тайм-трекер на Java». Отчёт о практическом занятии № 2. — 2026.